

DSA STUDY BLUEPRINT SERIES

116. Topological Sorting using Kahns Algorithm

Topological Sorting using BFS In-degree Counts (Kahn's)

Section 10 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Kahn's:** Compute in-degrees. Push in-degree == 0 nodes to queue; decrement neighbor values on pop.

Memory & Execution Blueprint

In-degree calculation: Node 0 (In-degree = 0) $\Rightarrow$
Process and push to queue.

C++ Implementation Reference

Standard code layout and syntax

```
vector topoBFS(int V, vector<int>& adj) {  
    vector<int> inDegree(V, 0), order;  
    for (int u = 0; u < V; u++) for (int v : adj[u]) inDegree[v]++;  
    queue<int> q;  
    for (int i = 0; i < V; i++) if (inDegree[i] == 0) q.push(i);  
    while (!q.empty()) {  
        int u = q.front(); q.pop(); order.push_back(u);  
        for (int v : adj[u]) if (--inDegree[v] == 0) q.push(v);  
    }  
    return order;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$

Key Practices & Gotchas

- **Important:** If output size is less than V , the graph has a cycle.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.